

Theory of Computation

(CSC262)

Unit 1: Basic Foundations

-Hemanta GC

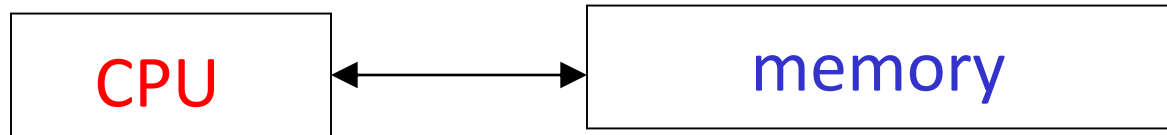
Fundamentals

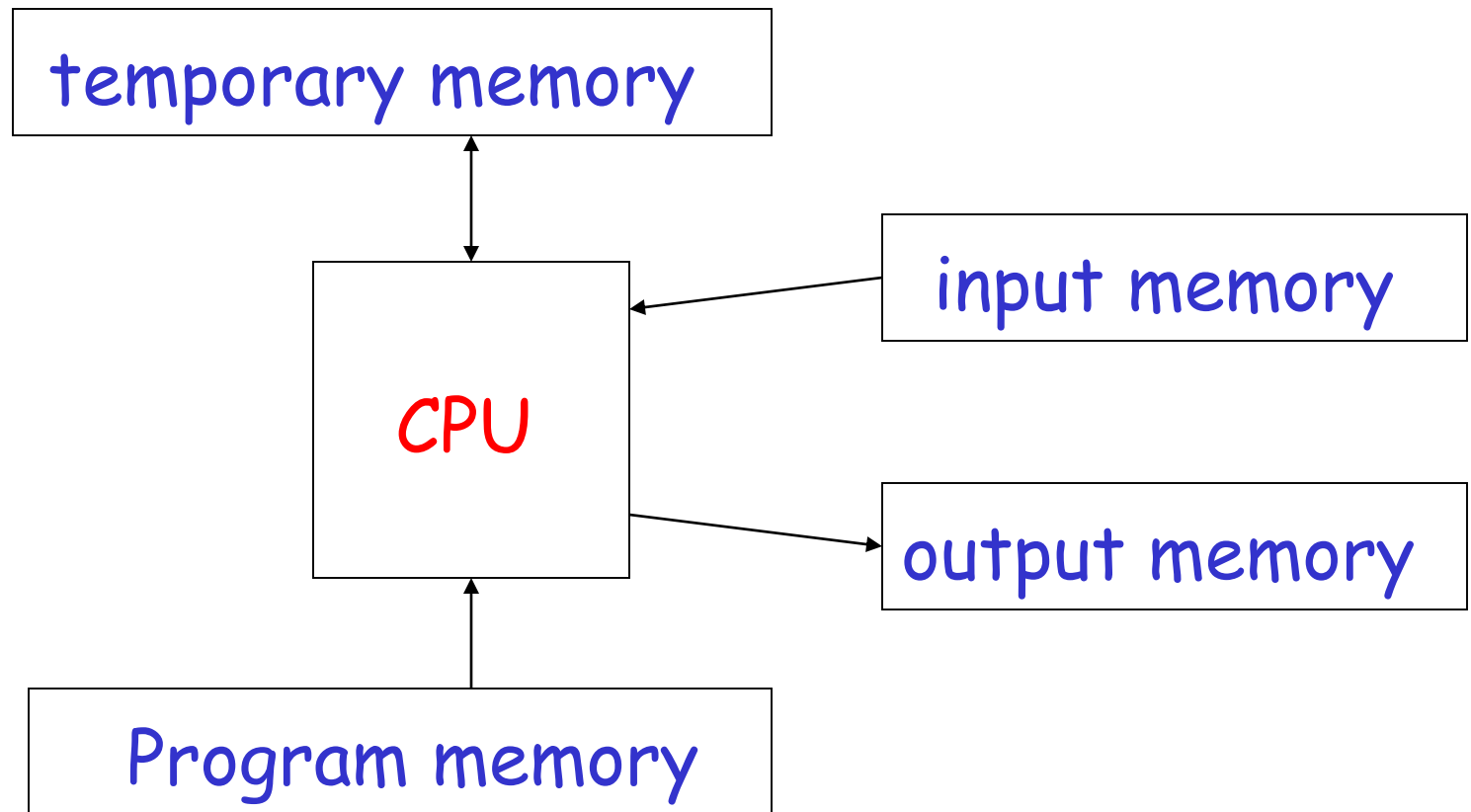
- **Automaton** = A self-operating machine or mechanism (Dictionary definition), plural is Automata.
- Automata = abstract computing devices
- Automata theory = the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems), and the computational problems that can be solved using these machines.
 - Mathematical models of computation
 - Finite automata
 - Push-down automata
 - Turing machines

Historical Aspect

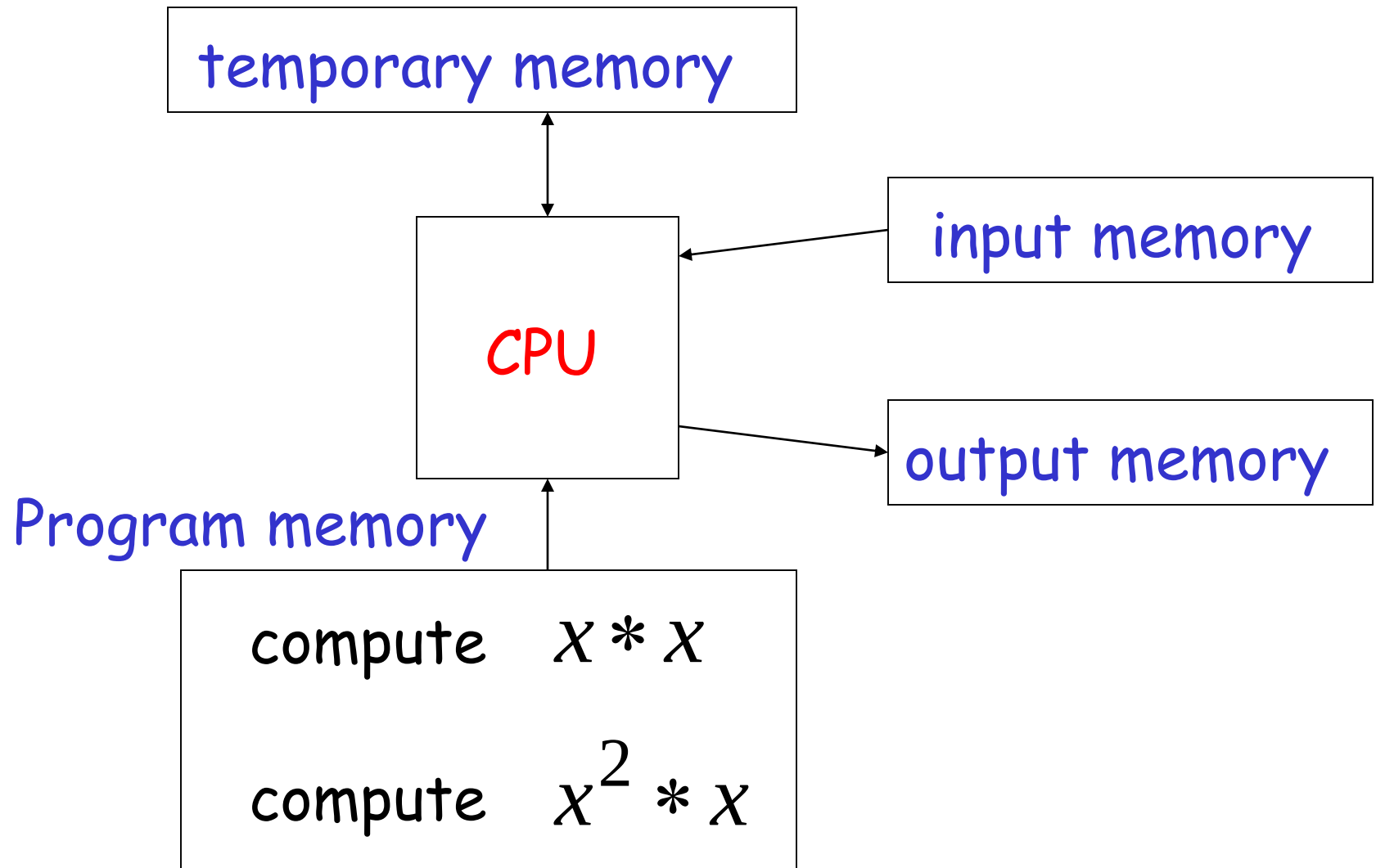
- Before 1930's, there were no computers, no any specific model, either real or abstract model of computation.
- 1930s : Alan Turing defined machines more powerful than any in existence, or even any that we could imagine – Goal was to establish the boundary between what was and was not computable.
- 1940s/1950s : In an attempt to model “Brain function” researchers defined finite state machines.
- Late 1950s : Linguist Noam Chomsky began the study of Formal Grammars which is closely related to abstract automata.
- 1960s : A convergence of all this into a formal theory of computer science, with very deep philosophical implications as well as practical applications (compilers, web searching, hardware, A.I., algorithm design, software engineering,...)

Computation

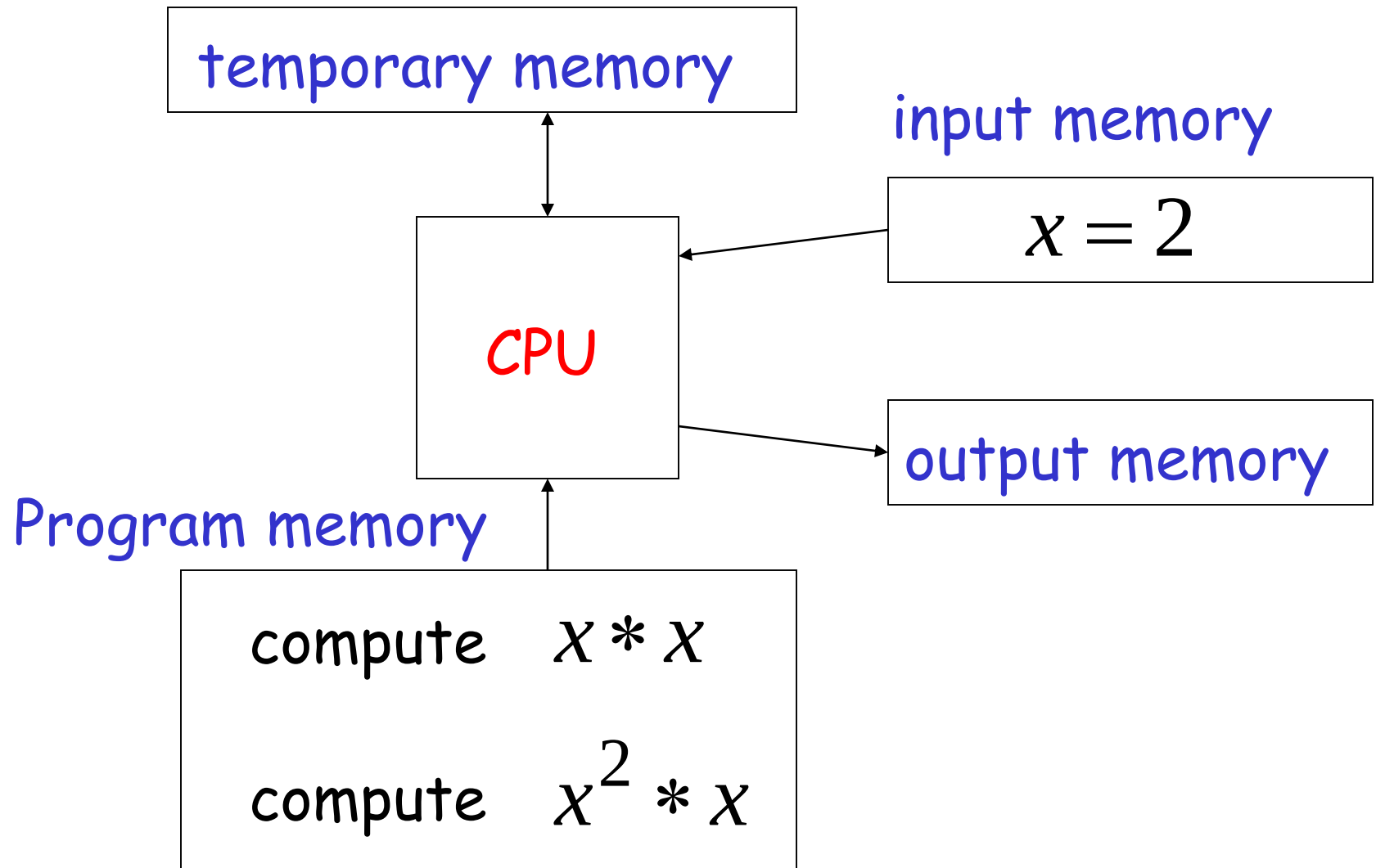




Example: $f(x) = x^3$



$$f(x) = x^3$$



temporary memory

$$z = 2 * 2 = 4$$

$$f(x) = z * 2 = 8$$

$$f(x) = x^3$$

input memory

$$x = 2$$

CPU

output memory

Program memory

compute $x * x$

compute $x^2 * x$

temporary memory

$$z = 2 * 2 = 4$$
$$f(x) = z * 2 = 8$$

$$f(x) = x^3$$

input memory

$$x = 2$$

CPU

$$f(x) = 8$$

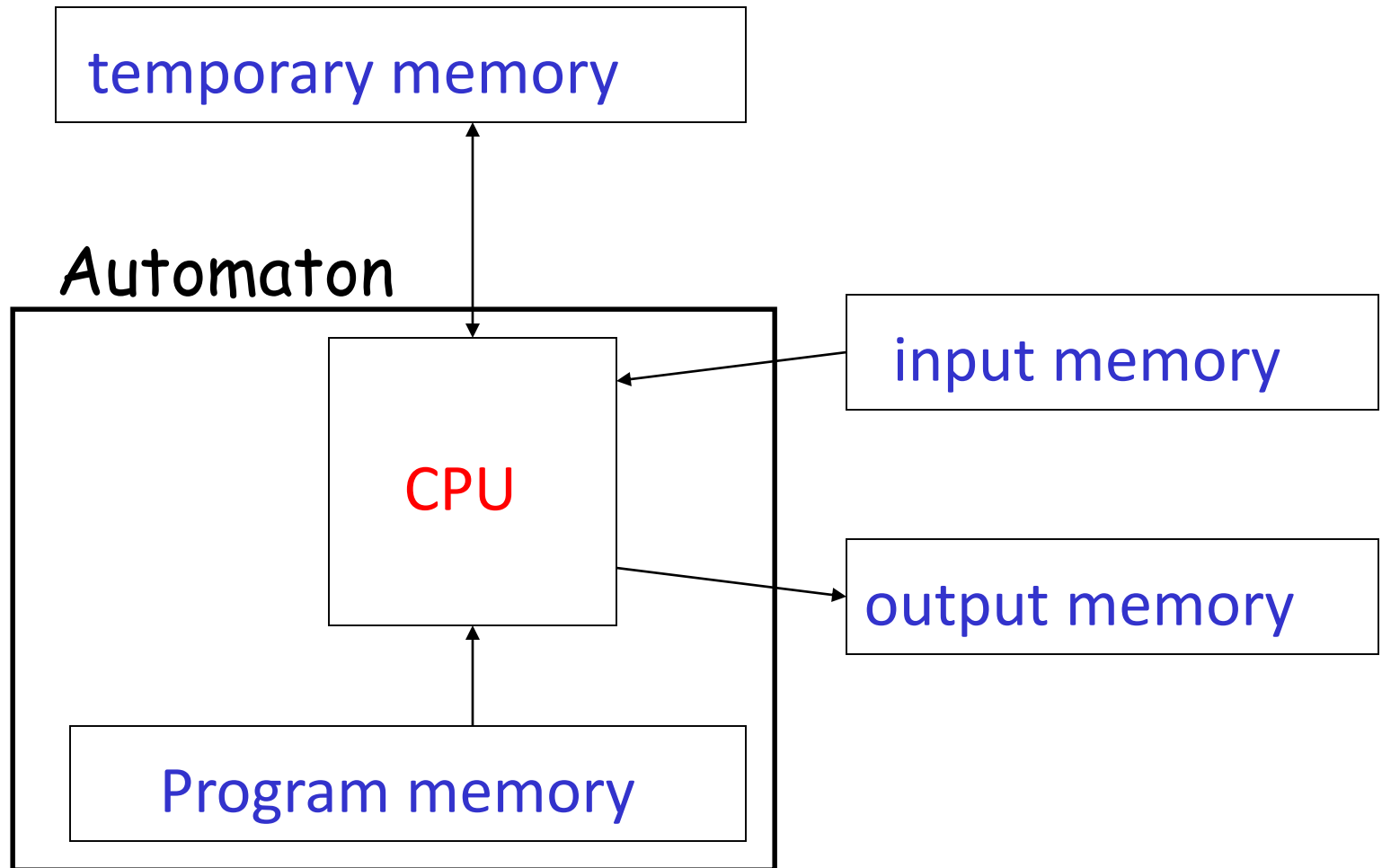
output memory

Program memory

compute $x * x$

compute $x^2 * x$

Automaton

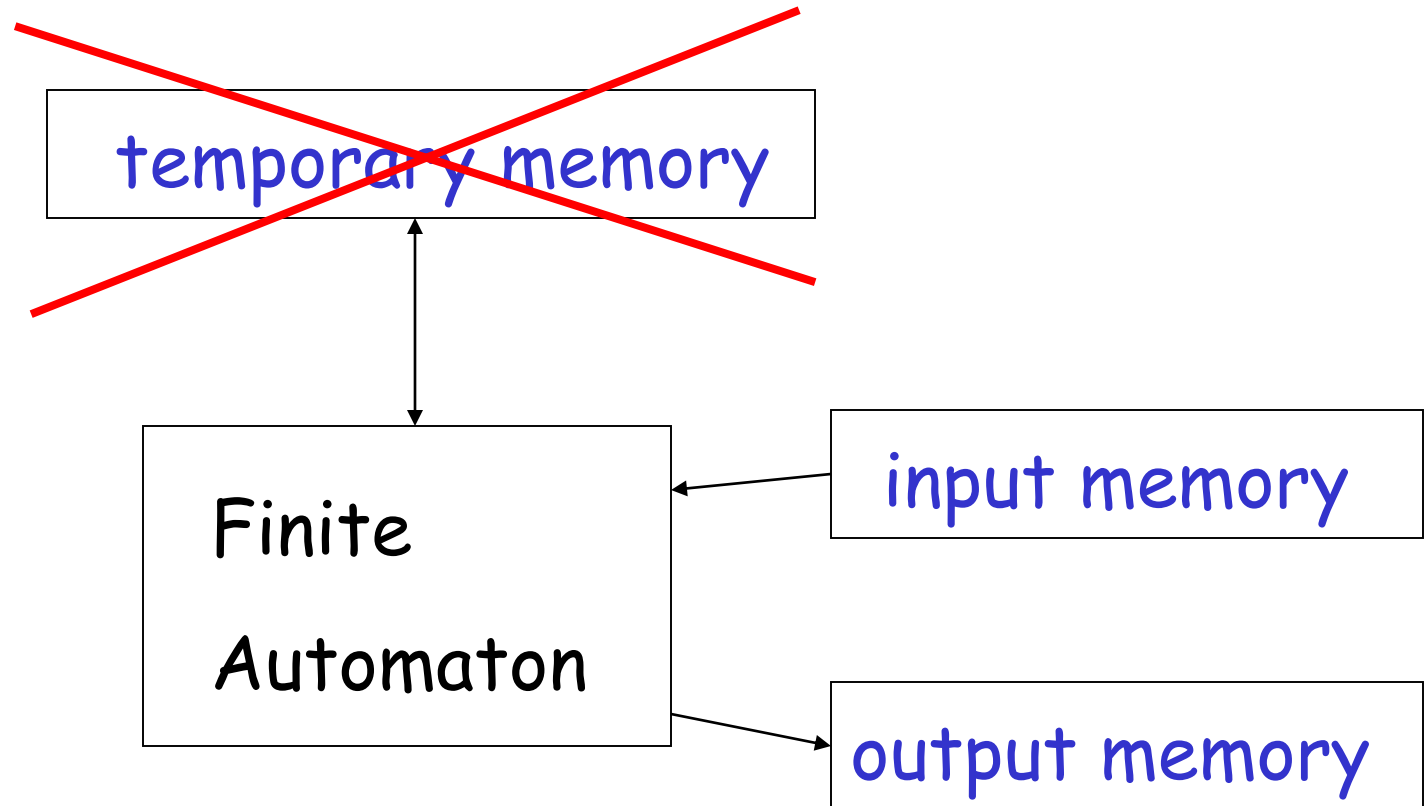


Different Kinds of Automata

Automata are distinguished by the temporary memory

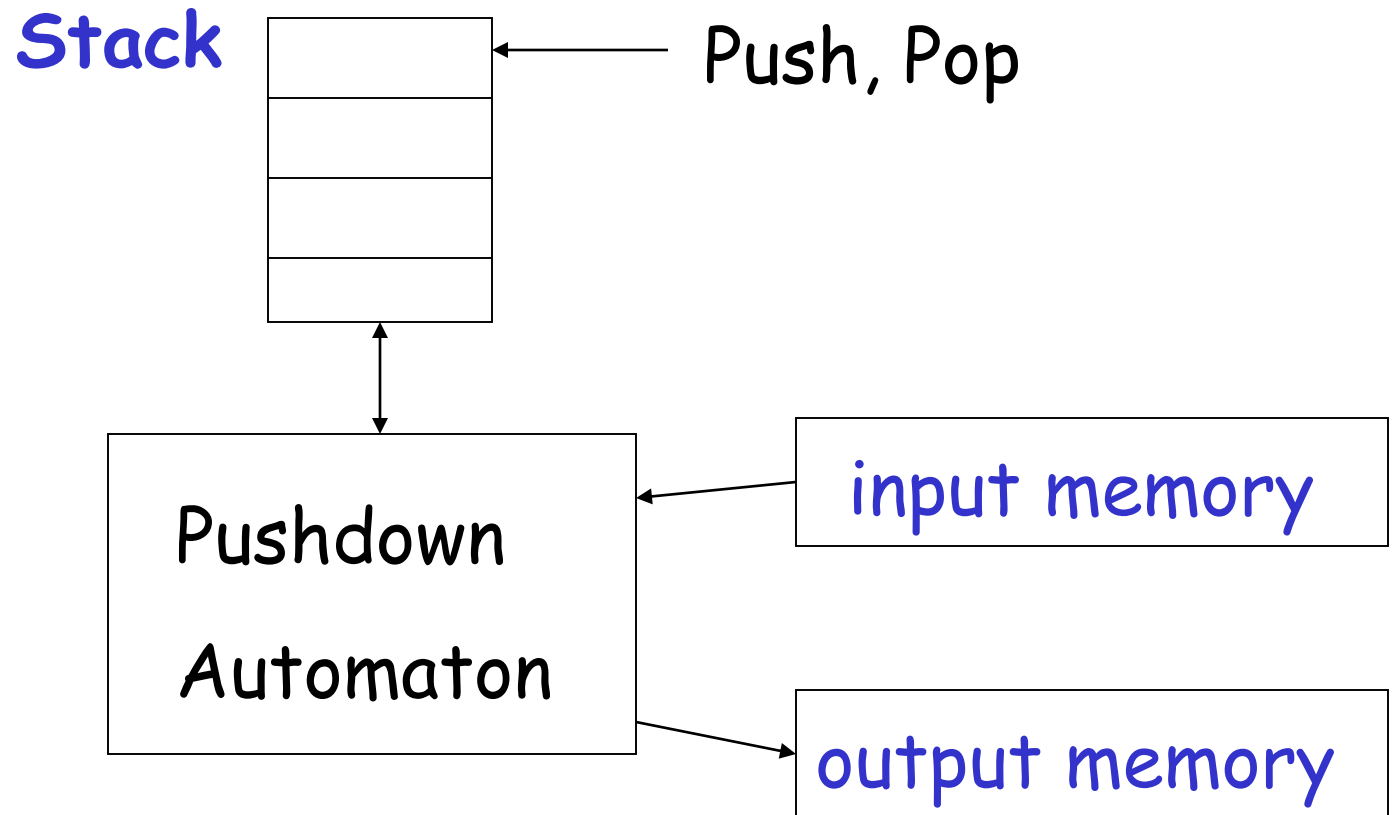
- **Finite Automata:** no temporary memory
- **Pushdown Automata:** stack
- **Turing Machines:** random access memory

Finite Automaton



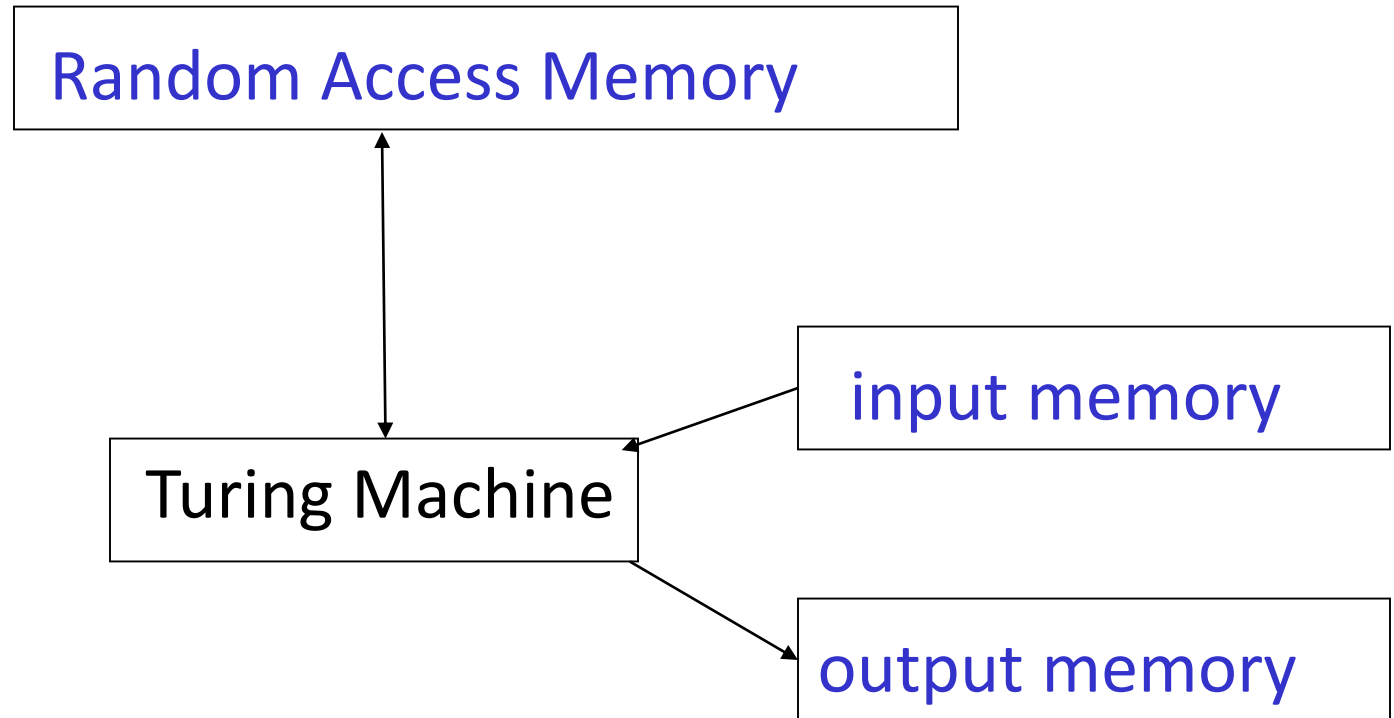
Example: Automatic Door, Vending Machines
(small computing power)

Pushdown Automaton



Example: Compilers for Programming Languages
(medium computing power)

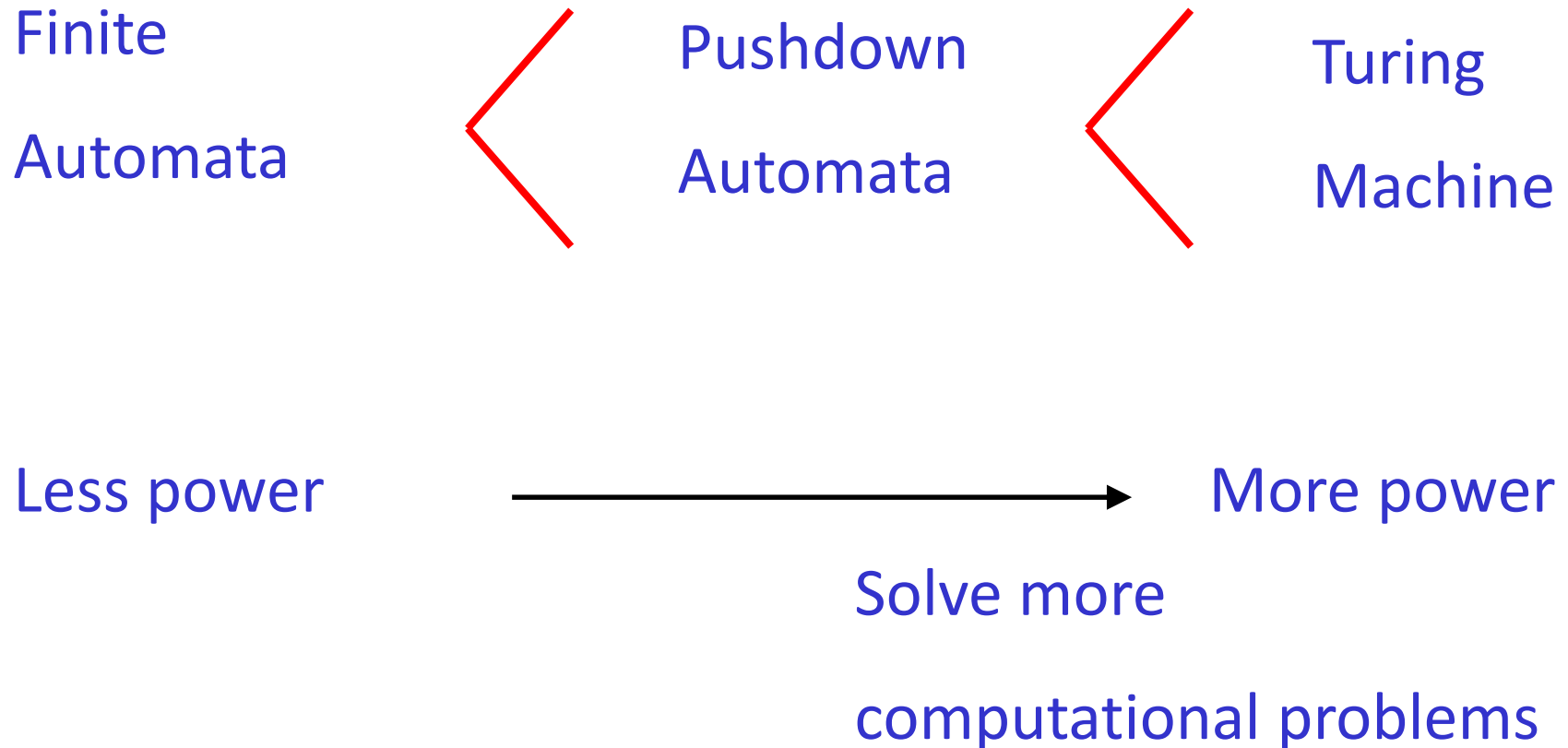
Turing Machine



Examples: Any Algorithm

(highest computing power)

Computing Power of Automata



Mathematical Preliminaries

Sets

Functions

Relations

Graphs

Proof Techniques

SETS

A set is a collection of elements

$$A = \{1, 2, 3\}$$

$$B = \{train, bus, bicycle, airplane\}$$

We write

$$1 \in A$$

$$ship \notin B$$

Set Representations

$$C = \{ a, b, c, d, e, f, g, h, i, j, k \}$$

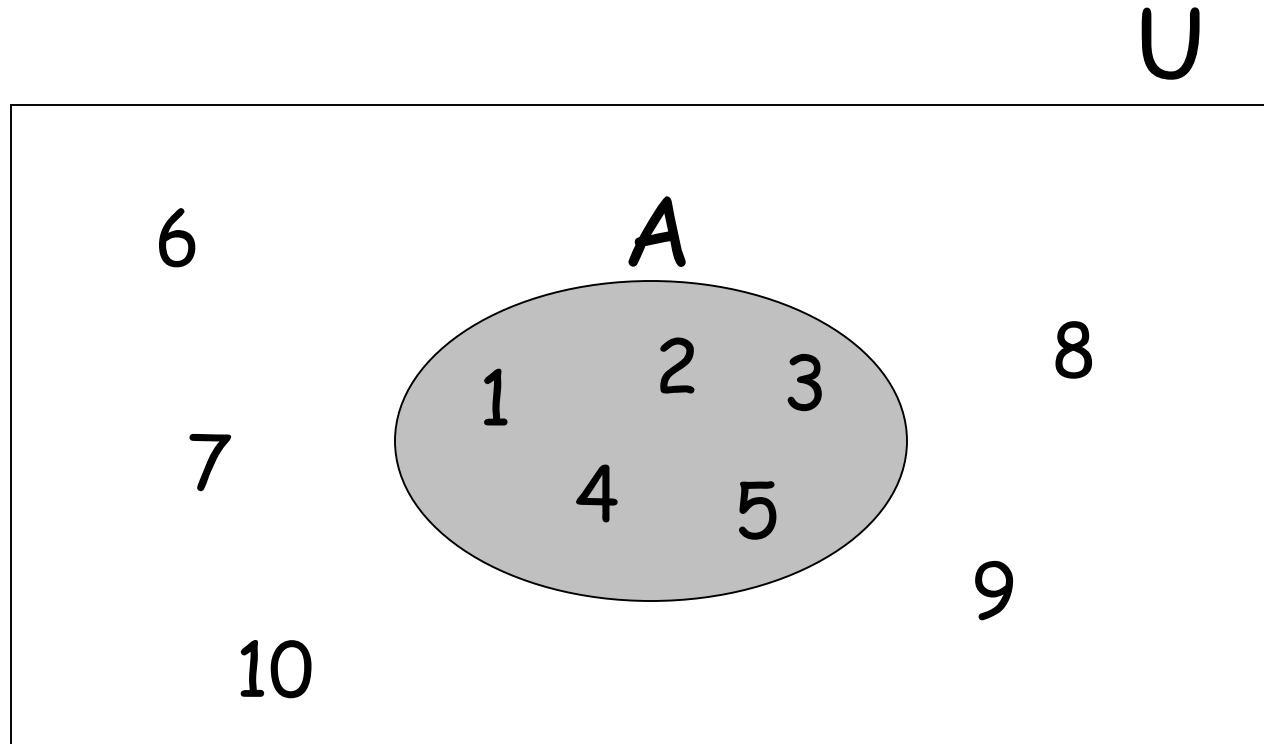
$$C = \{ a, b, \dots, k \} \longrightarrow \textit{finite set}$$

$$S = \{ 2, 4, 6, \dots \} \longrightarrow \textit{infinite set}$$

$$S = \{ j : j > 0, \text{ and } j = 2k \text{ for some } k > 0 \}$$

$$S = \{ j : j \text{ is nonnegative and even} \}$$

$$A = \{1, 2, 3, 4, 5\}$$



Universal Set: all possible elements

$$U = \{1, \dots, 10\}$$

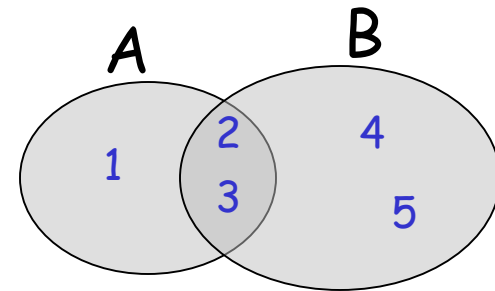
Set Operations

$$A = \{1, 2, 3\}$$

$$B = \{2, 3, 4, 5\}$$

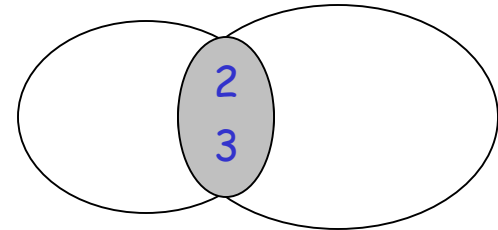
- Union

$$A \cup B = \{1, 2, 3, 4, 5\}$$



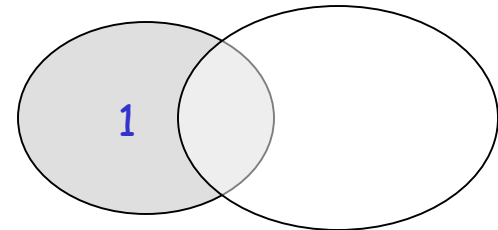
- Intersection

$$A \cap B = \{2, 3\}$$



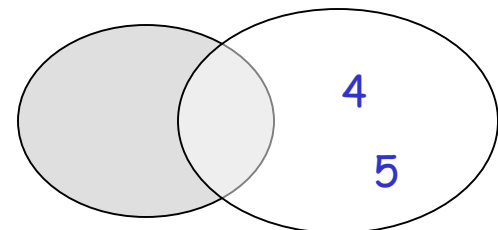
- Difference

$$A - B = \{1\}$$



$$B - A = \{4, 5\}$$

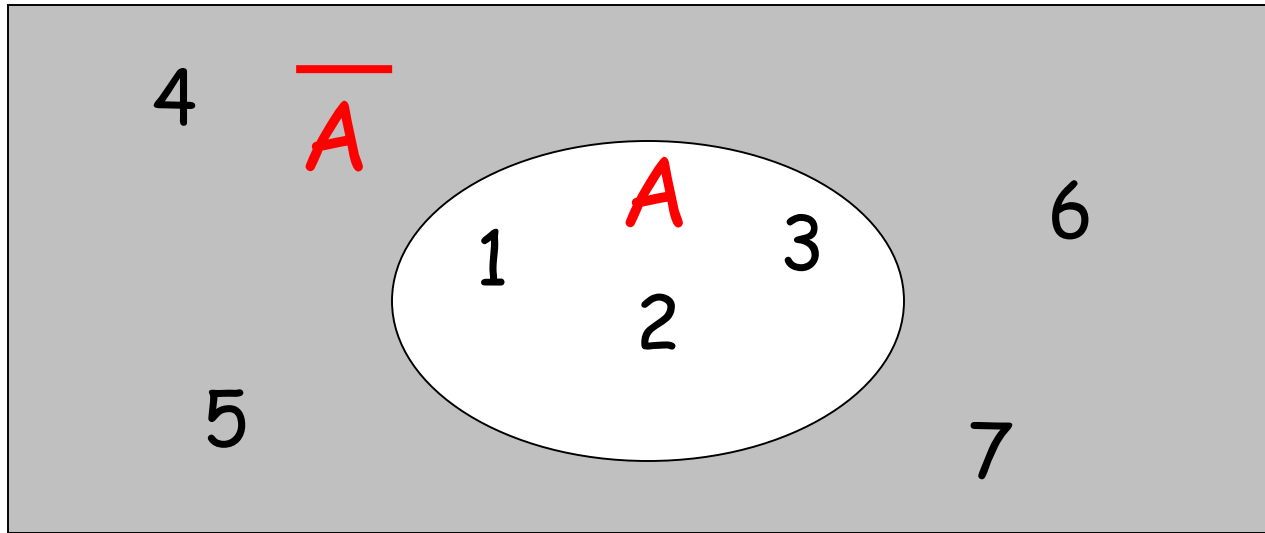
Venn diagrams



- Complement

Universal set = $\{1, \dots, 7\}$

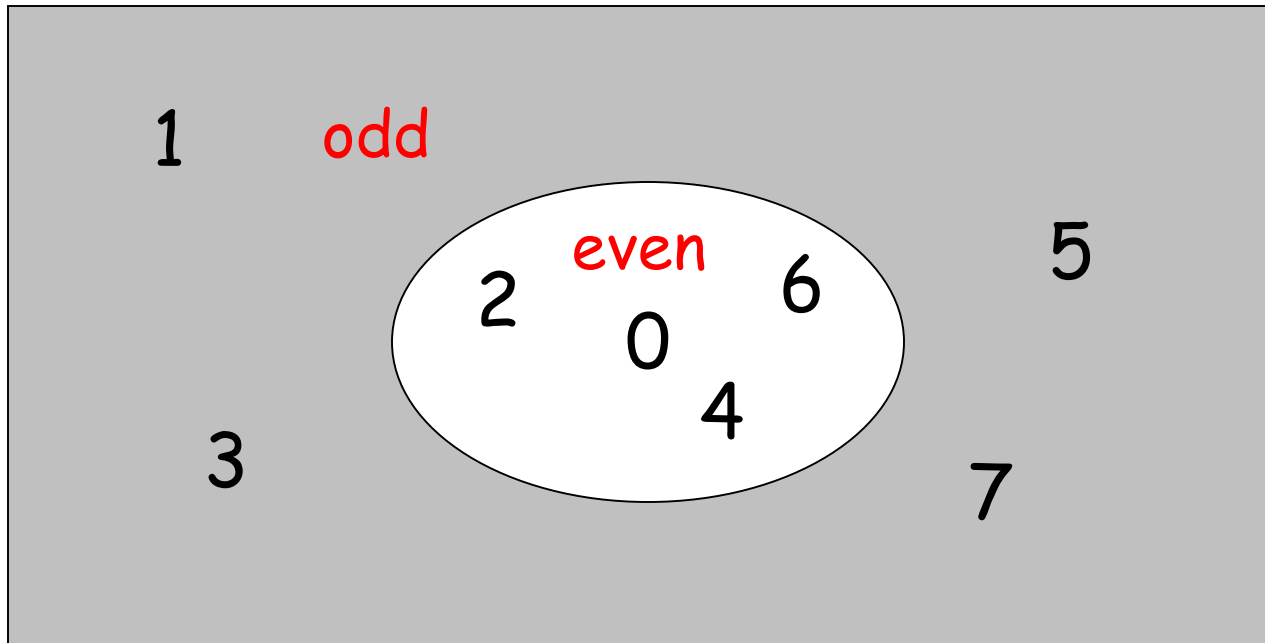
$$A = \{1, 2, 3\} \longrightarrow \overline{A} = \{4, 5, 6, 7\}$$



$$\overline{\overline{A}} = A$$

$$\{ \text{even integers} \} = \{ \text{odd integers} \}$$

Integers



DeMorgan's Laws

$$\overline{A \cup B} = \bar{A} \cap \bar{B}$$

$$\overline{A \cap B} = \bar{A} \cup \bar{B}$$

Empty, Null Set: $\emptyset$

$$\emptyset = \{\}$$

$$S \cup \emptyset = S$$

$$S \cap \emptyset = \emptyset$$

$$S - \emptyset = S$$

$$\emptyset - S = \emptyset$$

$$\overline{\emptyset} = \text{Universal Set}$$

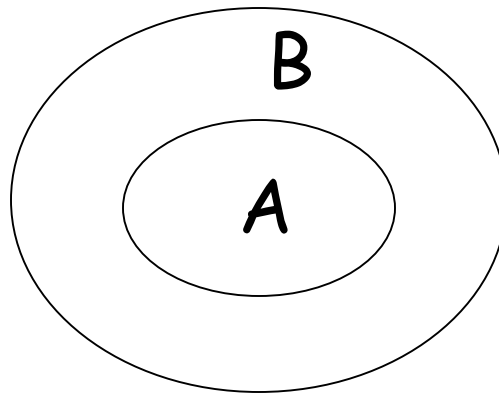
Subset

$$A = \{ 1, 2, 3 \}$$

$$B = \{ 1, 2, 3, 4, 5 \}$$

$$A \subseteq B$$

Proper Subset: $A \subset B$

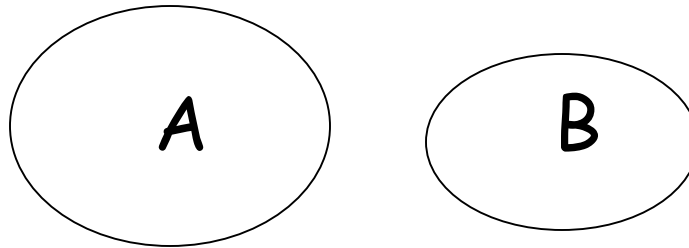


Disjoint Sets

$$A = \{ 1, 2, 3 \}$$

$$B = \{ 5, 6 \}$$

$$A \cap B = \emptyset$$



Set Cardinality

- For finite sets

$$A = \{ 2, 5, 7 \}$$

$$|A| = 3$$

(set size)

Power sets

A power set is a set of sets

$$S = \{ a, b, c \}$$

Power set of S = the set of all the subsets of S

$$2^S = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} \}$$

Observation: $|2^S| = 2^{|S|} \quad (8 = 2^3)$

Cartesian Product

$$A = \{ 2, 4 \}$$

$$B = \{ 2, 3, 5 \}$$

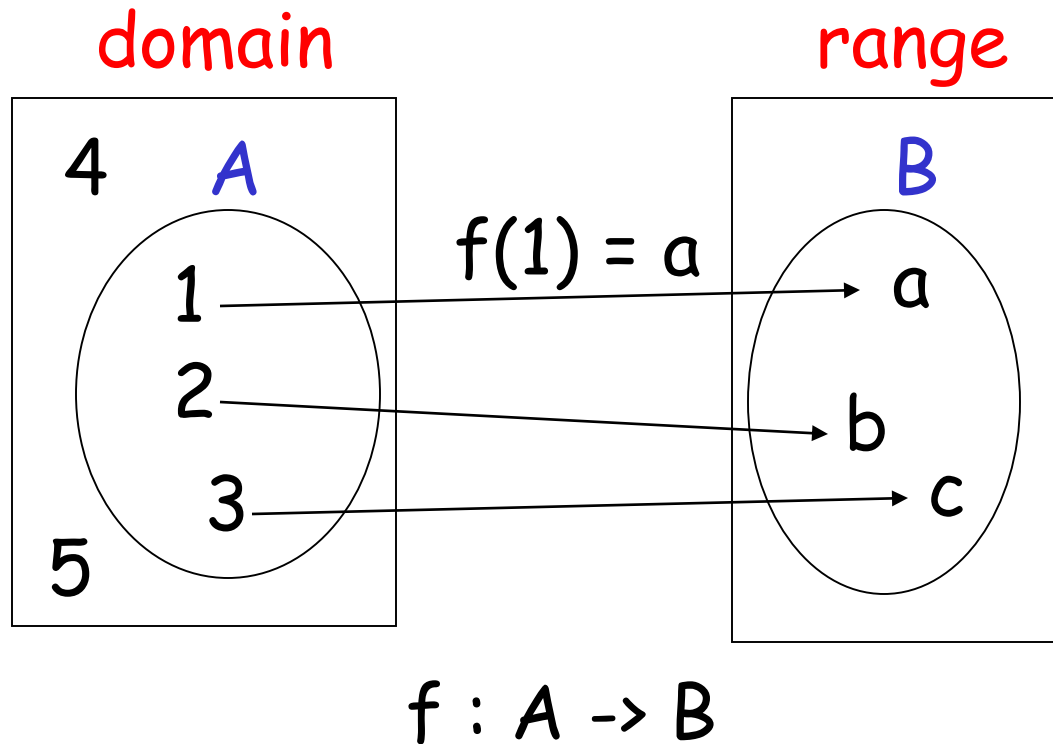
$$A \times B = \{ (2, 2), (2, 3), (2, 5), (4, 2), (4, 3), (4, 5) \}$$

$$|A \times B| = |A| |B|$$

Generalizes to more than two sets

$$A \times B \times \dots \times Z$$

FUNCTIONS



If $A = \text{domain}$

then f is a total function

otherwise f is a partial function

RELATIONS

Let A & B be sets. A binary relation " R " from A to B

$$R = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots\}$$

Where $x_i \in A$ and $y_i \in B$

$$R \subseteq A \times B$$

$x_i R y_i$ to denote $(a, b) \in R$

e. g. if $R = '>'$: $2 > 1, 3 > 2, 3 > 1$

Automata, Computability and Complexity

Automata Theory:

- Studies **abstract machines** and the **languages** they recognize.
- It is the foundation layer of theoretical Computer Science.

Main Models

- Finite Automata (FA)
- Pushdown Automata (PDA)
- Turing Machines (TM)

Main Goal : To classify languages according to the power of machines that recognize them.

Real Applications

- Compiler design
- Regular expressions
- Text Searching and processing
- Protocol verification – in communication

Automata, Computability and Complexity

Computability Theory: :Asks

- “Is this problem solvable by any algorithm whatsoever?”
- It studies the fundamental limits of computation.

Main Goal : To separate-

- Problems that are computable (decidable)
- Problems that are non-computable (undecidable)

Central Model:

- Turing Machine

Famous Problems:

- Halting Problem - Undecidable, no computer can ever solve
- SAT Problem - Decidable
- Post Correspondence Problem

Key Idea:

- **Computability Theory is about possibility**

Automata, Computability and Complexity

Complexity Theory: Assumes a problem *is computable* and asks:

- “How much computational resource is needed?”

Resources:

- Time
- Memory (space)

Main Goal:

- To classify problems by efficiency.

Example Questions:

- Can sorting be done faster?
- Is SAT efficiently solvable?
- How much memory does an algorithm require?

Key Idea

- **Complexity Theory is about efficiency.**

Automata, Computability and Complexity

Relationship :

- A useful way to think about the progression:

Automata Theory : Defines computational models.

Computability Theory : Determines what those models can or cannot compute.

Complexity Theory: Measures how efficiently they compute.

An Example and analogy: Imagine solving puzzles

- **Automata Theory:** What kind of puzzle-solving machine do you have?
- **Computability Theory:** Is the puzzle solvable at all?
- **Complexity Theory:** How long will it take to solve?

Distinction

- **Automata Theory → models of computation**
- **Computability Theory → limits of computation**
- **Complexity Theory → efficiency of computation**

Why Study Automata Theory?

- The study of automata and complexity is an important part of computer science. The motivation towards the study of automata theory can be summarized as:
 - Automata theory plays an important role when we are making software for designing and checking behavior of digital circuit.
 - The "*lexical analyzer*" of a typical compiler, that is, the compiler component that breaks the input text into logical units called "*tokens*".
 - Software for scanning large bodies of text, such as collections of web pages, to find occurrence of words, phrases or patterns(*searching*).
 - Automata theory is key to software for verifying systems of all types that have a finite number of distinct states, such as communication protocols, protocols for secure exchange of information.
 - Automata Theory is most useful concept of software for *natural language processing*.

Definitions and preliminaries

- **Alphabet**: An alphabet is a finite, non-empty set of symbols, that make up the language of concern.

e.g. $A = \{0, 1\}$ defines a Binary Alphabet

$B = \{+, -, \times, \div, \Gamma, \alpha\}$ - Alphabet of 6 symbols.

$C = \{\text{All ASCII Symbols}\}$ etc.

- For convention, we shall use Σ in this course for alphabet.

- **String** : (or word): A string is a finite sequence of symbols taken from an alphabet. For example,

011010, 00, **11**, 01, **10**, 1, 0 are strings from binary alphabet $\{0, 1\}$

asdfg – A string of lower case English alphabet.

10+5×3÷6 – string of digits and mathematical operator symbols from alphabet $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, \times, \div\}$

Definitions Contd...

- The ***length*** of a string w , denoted by $|w|$, is the no of symbols in w .
 - e.g. $w=010010$, $u=10+5\times3\div6$
 - $|w| = 6$ $|u| = 8$
- **Empty string**: It is a string consisting of zero occurrences of symbols from an alphabet. i.e. the empty string has length zero. It is denoted by ϵ so length $|\epsilon| = 0$.
 - *So empty string is a string obtained from the symbols of any alphabets.*

Definitions Contd...

- **Power of an Alphabet:** The set of all strings of a certain length say k , from an alphabet is the k^{th} power of that alphabet.

$$\text{i.e. } \Sigma^k = \{w \mid |w| = k\}$$

- e.g. Let $\Sigma = \{0,1\}$, then,
- $\Sigma^0 = \{\epsilon\}$
- $\Sigma^1 = \{0,1\}$
- $\Sigma^2 = \{00,01,10,11\}$
- $\Sigma^3 = \{000,001,\dots,111\}$ and so on.

Definitions Contd...

- **Kleene Closure**: The set of all strings(strings with any length) over an alphabet Σ , denoted by Σ^* is called *Kleene closure* of Σ . i.e. *Kleene Closure* is set of all strings over alphabet Σ with length 0 or more. Mathematically we can write,

$$\bullet \Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \quad \text{An Infinite set}$$

- **Positive closure**: The set of all strings from an alphabet Σ . except empty string is called positive closure of Σ . and denoted by Σ^+

- i.e.

$$- \Sigma^+ = \Sigma^* - \Sigma^0 = \Sigma^1 \cup \Sigma^2 \cup \dots \quad \text{An Infinite set of strings from } \Sigma \text{ except empty string}$$

Definitions Contd...

- **Suffix of string**: A string s is called a suffix of string w if it is obtained by removing zero or more leading symbols in w .
e.g. $w = \alpha\beta\gamma\delta$; $s = \beta\gamma\delta$, is a suffix of w .
Let $w = \text{apple}$, then all suffix of w are (apple, pple, ple, le , e , ϵ)
– s is proper suffix since $s \neq w$.
- **Prefix of a string**: The string s is called a prefix of string w if it is obtained by removing zero or more trailing symbol from w .
e.g. $s = \alpha\beta\gamma\delta$, $s = \alpha\beta\gamma$, is a prefix of w .
Let $w = \text{apple}$, then all prefix of w are { apple, appl, app, ap,a , ϵ)
– s is proper prefix since $s \neq w$.
- Look above in both suffix and prefix example, ϵ is both suffix and prefix of any string.

Definitions Contd...

- **Substring**: The string s is called a substring of w if it is obtained by removing zero or more leading or trailing symbols from w . It is proper substring if $s \neq w$.
- e. g. $w = abcdefgh$, Here the strings ab , bc , $cdef$, fgh are the substrings of w .
- ***Let $w = patan$***

The Valid Substrings:

- *All prefixes and suffixes of the strings*
- *More like ata , at , ta*
- *Not valid substrings: pn , aa , pt , aan , pan etc*
- **String concatenation**: means joining two or more strings together to form a single string. “hello”+”world” = helloworld

Definitions Contd...

- **Language**: A language L over an alphabet Σ is a subset of the set of all strings that can be formed out of Σ . i.e. A language is the subset of *Kleene closure* over an alphabet Σ . i.e. $L \subseteq \Sigma^*$.

Formally, $L = \{w \mid \text{something about } w\}$

- A language may be empty: $L = \varnothing$.
- The language $L_1 = \{\epsilon\}$ is not an empty language since it contains one string ϵ . So L is empty but L_1 is a language of empty string.
- A string over an alphabet Σ is any string $w \in \Sigma^*$.
- Σ^* is the language over Σ consisting of all strings.
- Any string that can be made from alphabet Σ is in Σ^* .

Definitions Contd...

Some examples of language:

- *English Language is a language from finite set of $\{A..Z, a..z\}$ alphabet which consists of the collection of legal English words(strings) from its alphabet.*
- *A programming language like C consists of legal C strings from its alphabet(ASCII character set)*
- Set of all strings over $\Sigma = \{0,1\}$ with equal no of 0's&1's.
 $L=\{\epsilon, 01, 0011, 10, 1100, 0101, 1010, \dots\}$
- $\varnothing$, the empty language, is a language over any alphabet.
- $\{\epsilon\}$ is the language consisting of only empty string and also a language from any alphabet.
- Set of all strings over binary alphabet with length of each exactly 2 is $\{00, 01, 10, 11\}$ **Formally , $L = \{w \mid w \in \{0,1\}^* \text{ and } |w|=2 \}$**
- Set of all strings over binary alphabet with length at most 2 is $\{\epsilon, 0, 1, 00, 01, 10, 11\}$ **Formally $L=\{w \mid w \text{ is in } \{0,1\}^* \text{ and } |w| \leq 2\}$**

Definitions Contd...

- **Problem:** A problem in automata theory is the question of deciding whether a given string is a member of some particular language.
- In other words, if Σ is an alphabet and L is a language over Σ , then problem is.
 - Given a string w in Σ^* , decide whether or not w is in L .

Types of Problems

- Decision Problem
 - When the answer are simply Yes(Accept) or No(Reject) then such problem is decision problem. E.g $S=\{a,e,i,o,u\}$, Is x a member of S?
- Search Problem
 - A solution algorithm must search for the correct structure to return. The return value by the algorithm may be single or structured. e.g. Searching text in web
- Optimization Problem
 - When the return value optimizes some objective function, such problem is optimization problem. e.g. finding shortest path in graph rather than any path
- Enumeration Problem
 - When the answer is list of all satisfactory structures, it is called enumeration problem. e.g. return all paths from A to B
- Counting Problem
 - When the result is the count of such structures rather than a list, we called it counting problem. E. g. return no of possible paths from A to B.

- Thank You